Source: docs\RAZORPAY_TEST_ACCEPTANCE_AUDIT.md | Export: 11 November 2026

Actual Razorpay TEST payment acceptance audit

Audit date: 8 November 2026. Target: `parinaya-silks`, Firestore `(default)`, Production `https://www.parinayasarees.com`.

This audit supersedes earlier readiness snapshots that reported missing configuration. The deployed application was identified as commit `f6448b02528d43a421a76efd0e81e525e35f8a8f`, deployment `dpl_4ouMZyZqkvAGb6HAZtVuxeQ4xJq8`. The changes described below remain local.

Conclusion

The supplied actual **Razorpay TEST** card transaction is captured and correctly confirmed in production application records. This is stronger evidence than mocked tests or a worker HTTP 200. It does not establish LIVE-payment acceptance, bank settlement, or completion of the remaining hosted test matrix.

No payment, refund, authenticated reconciliation/worker request, production write, configuration change, commit, push or deployment was initiated during this audit. Production inspection used scoped reads and sanitized Vercel logs. Automated mutations were confined to loopback emulators.

Transaction and earlier failure

Identifier	Verified mapping
Internal order	`270cf7d613cf9d181516427eeb2b9666`
Razorpay order	`order_TxVJFCujFNRHOr`
Captured payment	`pay_TxVMBaa36pfPF7`
Amount / currency / method	100 paise / INR / card
Stored result	Order `Confirmed`, payment status `paid`, payment state `CAPTURED`, reservation `consumed`

Times below are IST. Application timestamps record observations, not necessarily the original bank-event time.

1. **07:13:57:** order created; inventory reservation changed 10 units to 9.
2. **07:15:05:** a genuine `payment.failed` webhook for the same Razorpay order was accepted. Its payment ID differs from the later captured payment.
3. **07:15:23:** the application recorded `FAILED`. Subsequent reconciliation retained the active reservation, allowing the existing provider order to be retried.
4. **07:16:54:** `payment.authorized`, `payment.captured` and `order.paid` were accepted.
5. **07:17:06:** Vercel logged successful Checkout callback-signature verification.
6. **07:17:11:** the application recorded capture, confirmed the order, and consumed its existing reservation. The worker processed all four stored events by **07:17:19**.

`failedAt` is deliberately preserved as history; it is not the current outcome. The supplied Dashboard screenshot describes a cancelled attempt, but its identity/reason was not independently matched to the failed event through the provider API. The supported conclusion is a failed attempt followed by a successful capture on the same provider order.

The `payment.authorized` event exists, but `authorizedAt` is absent. Reconciliation fetched the current provider state after it had already become captured; settlement records an authorization timestamp only when it actually observes `AUTHORIZED`. It does not invent one. The local Admin labels now distinguish a missing authorization time from an unconfirmed payment, and an earlier failure from the current captured state. No production timestamps were changed.

Evidence and acceptance table

Production records means current trusted Firestore reads; **logs** means correlated deployed Vercel invocations; **screenshot** means owner-supplied evidence; **automated** means local synthetic providers/emulators, not another real transaction.

Check	Status	Evidence	Action required
Actual TEST capture, amount and mapping	PASS	Production order, attempt and provider-order mapping match the supplied IDs, 100 paise and INR; Dashboard screenshot shows TEST capture.	None for this transaction.

Check	Status	Evidence	Action required
Checkout callback verification	PASS	Actual `payment_callback_signature_verified` log; deployed code validates stored order ID plus payment ID, then fetches provider evidence and checks amount/currency.	None for this transaction.
Authorization event / captured state	PASS	Actual authorized and captured event records are processed; final state is captured.	Do not fabricate the missing authorization observation timestamp.
Earlier failure explained	PASS	Same provider order, different failed payment, active reservation retained, later single confirmation.	Provider-specific cancellation reason remains unverified.
No duplicate application order/payment binding	PASS	One order for this provider order, one for the captured payment, one payment attempt, one confirmation history entry.	None.
Exhaustive provider-side duplicate-charge search	NOT VERIFIED	Application and supplied capture agree; no independent authenticated provider payment-list read was available.	Inspect this Razorpay TEST order's payment list privately.
Inventory reservation / consumption	PASS	One inventory audit: 10 → 9, delta −1 for quantity 1; no release audit. Capture consumes the reservation without another deduction.	None.
Genuine captured webhook delivered	PASS	Correlated `payment_webhook_accepted` log and processed event record; Vercel's HTTP-202-filtered log matches this captured event.	None.
Genuine webhook signature accepted	PASS	Deployed HMAC verifier runs before any event acceptance; the actual accepted event has a durable body hash and matching TEST key binding.	Original raw body/signature are not retained, so a fresh independent HMAC recalculation was not performed.
Actual failed / authorized / captured / paid events	PASS	All four correlated records are `processed`, with no remaining processing error.	None.
Seven supported webhook event paths	PASS	Source handles `payment.authorized`, `payment.captured`, `payment.failed`, `order.paid`, `refund.created`, `refund.processed`, `refund.failed`; signed automated coverage exercises all seven.	Genuine refund-event delivery still needs approved TEST refund acceptance.
Duplicate webhook and settlement safety	PASS	Transactional event-ID/body-hash deduplication; automated concurrent duplicates, ID conflicts, repeated reconciliation, single confirmation and stock checks.	Hosted duplicate/replay acceptance remains a separate test; it was not triggered here.
Coupon usage on this payment	NOT APPLICABLE	No coupon applied; discount is zero.	Run an approved hosted coupon test later.
Loyalty on this payment	PASS	Redeemed/expected/earned/restored/reversed all zero; exactly one Reserved ledger entry with all deltas zero. No duplicate deduction/allocation.	Nonzero redemption and delivery earning require separate hosted acceptance.
Failure, expiry and interrupted capture recovery	PASS	Actual failed-to-captured retry; automated trusted expiry release once, unresolved authorization hold, late capture review, lost-create response and capture-after-commit-failure recovery.	Complete hosted interruption/expiry scenarios with approved fixtures.
Worker authentication and operation	PASS	Unsigned worker rejected; actual authenticated worker invocations and heartbeat prove the deployed authorization path ran.	Maintain scheduler secret and alerting.

Check	Status	Evidence	Action required
Scheduler frequency	PASS	Supplied cron history shows one-minute scheduled executions; 24 sampled Vercel worker completion logs have median interval about 63 seconds.	Logs include variable delays; this is sampled evidence, not an uptime guarantee. Monitor failures and delays.
Heartbeat freshness / mode	PASS	At 08:12:35 IST, heartbeat was 22 seconds old, within the 300-second requirement; mode `test`, key binding matches this attempt. No due events or attempts at that read.	Keep monitoring freshness.
Zero worker counters	PASS	No due work on those invocations. Captured attempts are scheduled for later rechecks, so zero does not mean there are no stored attempts.	None; alert on persistent due work or errors instead.
Effective TEST credentials / allowlist	PASS	Actual authenticated initiation succeeded through the production UID and worker gates; genuine provider reconciliation and signatures succeeded. Unit/emulator tests reject nonapproved accounts and arbitrary client UID/amounts.	Maintain the minimal approved UID list privately. The list contents were not printed or read back.
LIVE currently inactive	PASS	Actual payment, heartbeat and key binding are TEST. Worker cannot change activation settings.	Keep `RAZORPAY_MODE=test`; do not activate LIVE as part of this audit.
Exact `RAZORPAY_LIVE_ACTIVATED=false` literal	NOT VERIFIED	Owner reports false; Vercel confirms a sensitive Production variable exists but does not reveal its value. TEST operation does not prove that separate literal.	Privately confirm it remains `false`.
Production variables / Preview isolation	PASS	All seven payment variables exist as sensitive Production-only entries; none scoped to Preview. Hosted browser-origin rejection and configuration tests fail closed.	Never copy credentials to Preview. Source does not impose a universal `VERCEL_ENV=production` lock if credentials are copied later.
Secrets in client / responses / sampled logs	PASS	Secret variables used server-side; built frontend has no server-secret variable names or private-key marker. Responses expose only the public checkout key and required public identifiers; sampled payment logs contain no secret/token values.	This is a scoped source/build/log review, not proof of every historical log or private dashboard setting.
Firestore and server authorization	PASS	Published rules match local rules. Admin requires claim and enabled access record; emulator coverage checks ownership and denied direct payment writes.	No rules publication required.
Customer hosted history/details presentation	NOT VERIFIED	Stored public projection is paid/captured and local browser coverage passes. Existing authenticated browser session could not be inspected.	Owner: refresh My Orders and this order's details; verify paid online status and no COD demand.
Admin hosted presentation	PASS	Supplied screenshot agrees with current production records. The timestamp-label clarification is local only.	Review/deploy that small copy change separately if desired.
Actual refund tracking	NOT VERIFIED	No refund on this order. Automated pending/failed/partial/full-refund cases pass; processed refunds cannot be downgraded.	Approved TEST refund acceptance is still required; no refund was initiated.
COD and nonzero coupon/loyalty regression	PASS	Existing local service and browser tests exercise trusted pricing, stock, ownership, COD collection, coupon eligibility/usage and Apply/Remove Points.	Hosted acceptance remains separate.

Webhook endpoint: **`https://www.parinayasarees.com/api/payments/webhook`**. The actual HTTP 202 acknowledges durable acceptance; asynchronous worker processing is verified separately above.

Local changes and verification

- `src/payments/AdminPayment.tsx`: clearer authorization-time and historical failure labels; no financial/state transition changes.
- `src/payments/security.test.ts`: rendered captured-payment regression for those labels.
- `scripts/payments-browser.test.py`: forwards blocked reviews without converting a null total; retains all payment assertions.
- `scripts/checkout-design-browser.test.py`: captures real review JSON before forwarding its response, avoiding a navigation/resource-disposal race; retains authoritative-total assertions.
- `scripts/payments-emulator.test.ts`: all seven signed event paths, concurrent duplicate delivery, processed records, single confirmation, unchanged stock and no invented refunds.
- This audit report. Existing `.env.example`, `docs/PAYMENTS.md` and `docs/RAZORPAY_TEST_SETUP.md` working-tree edits were preserved.

Checks actually completed:

- `npm run test`: **186 passed**, zero failures/skips.
- `npm run lint`: TypeScript passed.
- `npm run build`: production frontend build passed.
- `npm run test:catalog-runtime` and `npm run test:payments-runtime`: passed, including server-function packaging/runtime and original webhook byte handling.
- Ten relevant emulator/rules suites passed: payments, orders, coupons, loyalty, delivery-loyalty, shopping, delivery, Admin rules, customer rules and category rules. The expanded payment suite was rerun separately: **16 passed**, including its parent test and all 15 subtests.
- Five browser suites passed: payments, checkout design, checkout coupons, checkout context and COD. Payment/checkout design were rerun after fixing response-observation and teardown races: both passed with no Python traceback or route-teardown error. Payment widths: 320/390/768/1440; checkout additionally 1920; COD exercises six widths.
- `git diff --check`: passed. Built JavaScript contains none of the three server-secret variable names or a private-key marker.

The initial payment browser mock incorrectly converted a blocked review's null total. The checkout observer also raced with navigation and later teardown. These were test-harness issues, corrected without weakening assertions. The first new seven-event fixture used invalid event-ID punctuation; it was corrected to the supported identifier format and passed. No checks were disabled or assertions removed to obtain a pass. The broader unrelated browser suite was not rerun.

Remaining hosted acceptance plan - requires approval before execution

Use only an allowlisted existing customer and owner-approved disposable TEST stock/coupon/points fixtures. Razorpay TEST isolates money, **not this production Firestore database**. Capture before/after counts without sharing customer details. Do not induce a general production outage or stop the shared scheduler to simulate failure.

Scenario	Procedure / required result
Success	Supplied actual card success already passed. Also exercise each intended available method, particularly UPI and net banking, using the current official TEST methods. Check one internal/provider order, exact amount, captured state and one stock reservation.
Failure	Use Razorpay's official mock failure path. No confirmed order; reservation stays held until trusted expiry/recovery. Failure must not be presented as a refund.
Close Checkout	Dismiss before success. Customer sees pending/verification guidance, never success. Check status and resume the existing attempt.
Retry	Resume after failed/dismissed payment. Reuse the same internal and provider order; one successful capture and one stock reservation.
Duplicate initiation / callbacks	Double-click and refresh using one operation ID; repeat callback locally first. No second order, stock deduction, coupon use or point reservation. Do not deliberately pay an already captured order again.
Duplicate webhook	Already covered locally. With approval use an available official TEST delivery/replay facility, then compare event IDs and settlement counts. Acknowledged events may not be replayable from Dashboard; do not assume a replay button can resend them.
Interrupted recovery	Close the customer tab after approved payment submission; let normal webhook/worker recover. Reopen the existing order. Confirm captured state and exactly-once effects.

Scenario	Procedure / required result
Refunds	With explicit approval, request a partial/full TEST refund in Razorpay Dashboard. Observe genuine refund events, pending/processed/failed status, exact cumulative amount and no automatic inventory restoration. Partial refunds with loyalty require trusted merchandise/shipping allocation review; no completed refund-allocation workflow is promised by this audit.
COD	With approved fixtures, place COD, verify due-on-delivery rather than online-paid, one deduction, fulfilment and separate external collection.
Coupons / loyalty	Apply/remove/reapply eligible coupon and points; verify server totals. Exercise success and trusted expiry/release. Deliver an eligible order once and verify one points award; capture alone must not award delivery points.

Official references: TEST cards and mock success/failure, webhook validation and deduplication, Dashboard replay limitations. Follow the current provider instructions and account-supported methods, not real card or bank credentials.

Before LIVE activation

1. Owner privately confirms the LIVE flag remains false during TEST; checks the provider TEST order/payment list, capture setting and all seven webhook subscriptions. No secret values need to be shared.
2. Complete the approved hosted matrix above, including genuine refund events, nonzero coupons/points, UPI/net banking if offered, customer order presentation and interruption recovery. Review delayed worker runs and add operational monitoring.
3. Review remaining application limitations, especially refund allocation/loyalty holds and manual fulfilment/accounting. Approve the operating procedure rather than assuming webhook receipt completes those workflows.
4. Separately confirm Razorpay account/KYC activation, enabled LIVE payment methods, settlement account, approved business/policy/contact information and operational ownership. This audit did not verify those dashboard/account facts.
5. Only with explicit approval: configure the correct LIVE key pair, separate LIVE webhook/secret, intentional Production-only mode/activation, updated worker binding and monitored release. Keep old TEST attempts bound to their original mode/key; they cannot simply be resumed under LIVE credentials. No LIVE transition was performed here.

Browser inspection was blocked because Computer Use could not reliably determine the Chrome URL. It was stopped; existing screenshots were not represented as a newly performed hosted UI test. No independent provider dashboard/API enumeration, original-body HMAC recomputation, refund or LIVE transaction was performed.